这是一个基于 **Kruskal 算法（构建最小生成树）** + **倍增法（求树上路径最大值）** 的标准实现方案。

为了让你看得懂，我把代码拆解为三个核心模块：

1. 数据结构与预处理

我们需要存储整棵树的倍增表 `up[u][i]` 和最大权值表 `max_e[u][i]` 。

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

const int MAXN = 100005;
const int LOGN = 18; // 2^18 > 100000

struct Edge {
    int u, v, w, id;
    bool in_mst; // 是否在最小生成树中
};

vector<pair<int, int>> adj[MAXN]; // MST的邻接表
int up[MAXN][LOGN];     // 2^i 级祖先
int max_e[MAXN][LOGN];  // 到 2^i 级祖先路径上的最大边权
int depth[MAXN];

// 倍增预处理
void dfs(int u, int p, int d, int w) {
    depth[u] = d;
    up[u][0] = p;
    max_e[u][0] = w;
    for (int i = 1; i < LOGN; i++) {
        up[u][i] = up[up[u][i - 1]][i - 1];
        max_e[u][i] = max(max_e[u][i - 1], max_e[up[u][i - 1]][i - 1]);
    }
    for (auto& edge : adj[u]) {
        if (edge.first != p) dfs(edge.first, u, d + 1, edge.second);
    }
}
```

2. 核心查询：在路径上跳跃（倍增的核心）

这部分就是你最关心的“如何通过倍增找到路径上的最大边”。

```
int query_max(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int res = 0;
    // 1. 先把 u 提上来，提到和 v 同一层
    for (int i = LOGN - 1; i >= 0; i--) {
        if (depth[u] - (1 << i) >= depth[v]) {
            res = max(res, max_e[u][i]);
            u = up[u][i];
        }
    }
    if (u == v) return res;
    // 2. u 和 v 一起向上跳，直到它们在 LCA 的下方
    for (int i = LOGN - 1; i >= 0; i--) {
        if (up[u][i] != up[v][i]) {
            res = max(res, max(max_e[u][i], max_e[v][i]));
            u = up[u][i];
            v = up[v][i];
        }
    }
    // 最后一步：还要把这两点连向 LCA 的那两条边考虑进去
    return max(res, max(max_e[u][0], max_e[v][0]));
}
```

3. 主逻辑：求次小生成树

```
// 假设 edges 是已经排好序的所有边
long long mst_weight = 0;
// ... 先跑一遍 Kruskal 算出 mst_weight ...
// ... 再调用一次 dfs(root, root, 0, 0) 构建倍增表 ...

long long ans = 2e18; // 设置一个无穷大
for (auto& e : edges) {
    if (!e.in_mst) {
        // e 是未被选中的边，尝试替换
        int max_w = query_max(e.u, e.v);
        // 如果 max_w > 0，说明存在替换方案
        ans = min(ans, mst_weight - max_w + e.w);
    }
}
cout << ans << endl;
```

代码逻辑复盘（对照你的疑问）：

1. **`query_max` 函数**：它就是利用 `max_e` 这个二维表，把 u 到 v 路径上每一个“ 2^i 跨度”的最大值都取出来对比。这就是**为什么不用遍历整个路径**的原因，因为每一段 2^i 跨度里的最大值我们早就预处理好了。
2. **`mst_weight - max_w + e.w`**：这就是公式 $M' = M + w - w'$ 的代码体现。我们遍历了所有不在 MST 里的边，对于每一个边，我们都算出了一个候选的 M'，最后取 `min` 得到的就是次小生成树。
3. **为什么不担心 LCA 被跳过？** 在 `query_max` 的第二个 `for` 循环中， `if (up[u][i] != up[v][i])` 保证了我们始终在向 LCA 逼近，但绝对不会跳过它。这就如同你之前担心的那样，它通过精准的二进制步长，把 u 和 v 卡在了 LCA 的正下方。

这段代码完全覆盖了你的所有疑惑点。你只需要把 Kruskal 的部分补全，就能直接运行了！怎么样，逻辑是不是清晰多了？